

Binary Numbers

ACOL215: Digital Logic And System Design

Abhilash Jindal

Digital systems

- Represent and manipulate *discrete elements* (set) of information. Example: *digits (hence digital systems)*, letters of alphabet, playing cards, chessboards squares
- Discrete elements are represented by electrical signals (voltages/current)
- Signals are controlled and manipulated by transistors

Discrete quantities of information

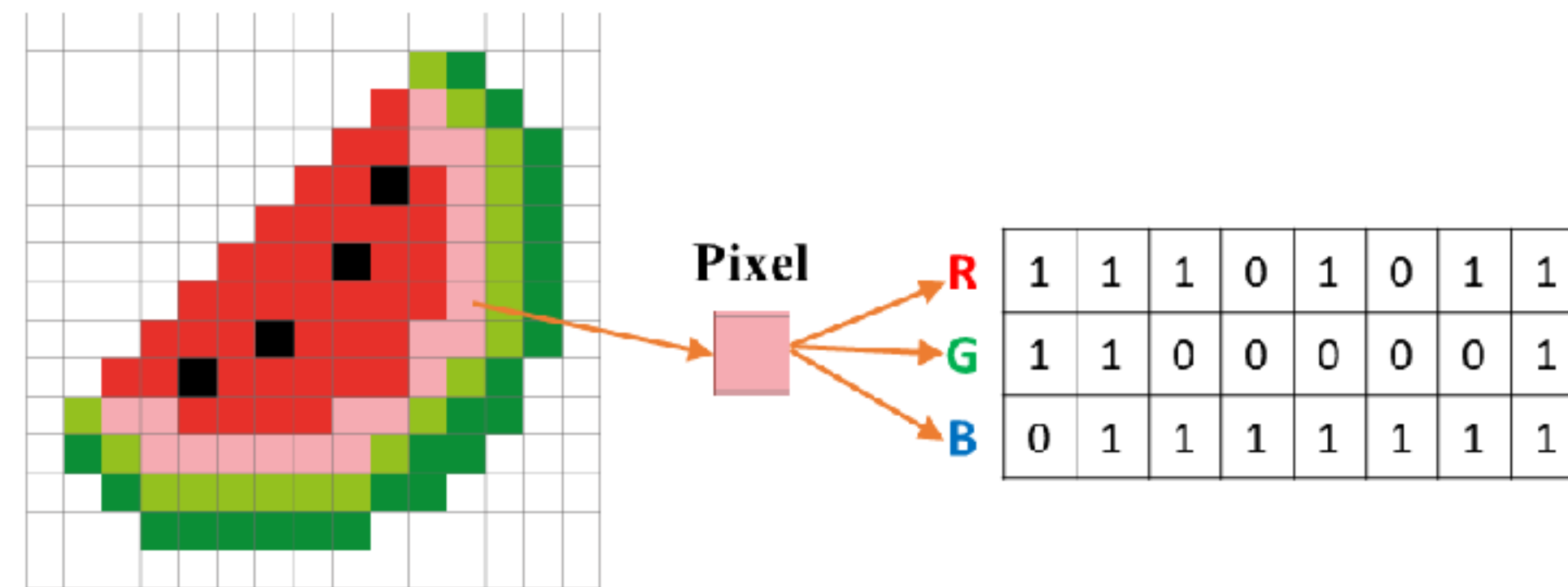
- Inherently discrete processes
 - Payroll management: Employee names, PAN Card number, monthly salaries, income tax deduction
- Continuous processes
 - Example: Digital camera.
 - Colors are discretised into numbers
 - Images are discretised into pixels, where each pixel has a discretised color

	Red	Green	Blue
	0	0	0
	255	255	255
	255	0	0
	0	255	0
	0	0	255
	255	128	0
	255	255	0
	128	128	128



Present-day digital systems

- Use just two discrete values: on<>off
- Binary digit (bit) has two numerical values: 0 or 1
- Discrete elements of information are represented with group of bits



Decimal numbers

How to represent numbers with bits?

- Consider 7,392.54
- $= 7000 + 300 + 90 + 2 + 0.5 + 0.04$
- $= 7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0 + 5 \times 10^{-1} + 4 \times 10^{-2}$
- In general, a number $\cdots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \cdots =$
 $\cdots + a_3 \times 10^3 + a_2 \times 10^2 + a_1 \times 10^1 + a_0 \times 10^0 + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} + \cdots$
- For 7,392: $a_3 = 7; a_2 = 3; a_1 = 9; a_0 = 2$. Rest are zero.

Radix

- A decimal number $\cdots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \cdots = \cdots + a_3 \times 10^3 + a_2 \times 10^2 + a_1 \times 10^1 + a_0 \times 10^0 + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} + \cdots$
- Decimal numbers have **10** symbols: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}.
- Each coefficient is multiplied with a power of **10**.
- Number is in base 10 or radix 10
- Written as $(7,392.54)_{10}$

Base-r number system

- Base-r numbers have **r** symbols: $\{0, \dots, r - 1\}$. $a_j \in \{0, \dots, r - 1\}$
- A base-r number $\dots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \dots =$
 $\dots + a_3 \times r^3 + a_2 \times r^2 + a_1 \times r^1 + a_0 \times r^0 + a_{-1} \times r^{-1} + a_{-2} \times r^{-2} + \dots$
- Each coefficient is multiplied with a power of **r**.

Examples

- Base-5 numbers have 5 symbols $\{0, \dots, 4\}$. $(4021.2)_5 = ?$
 - $4 * 5^3 + 0 * 5^2 + 2 * 5^1 + 1 * 5^0 + 2 * 5^{-1} = (511.4)_{10}$
- Octal numbers (base-8) have 8 symbols $\{0, \dots, 7\}$. $(127.4)_8$
 - $1 * 8^2 + 2 * 8^1 + 7 * 8^0 + 4 * 8^{-1} = (87.5)_{10}$

Binary numbers

- Binary numbers (base-2) have 2 symbols $\{0,1\}$. $(11010.11)_2 = ?$
 - $1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 + 1 * 2^{-1} + 1 * 2^{-2} = (26.75)_{10}$
- $(10101)_2 = ?$
 - $(21)_{10}$
- $(110101)_2 = ?$
 - $(53)_{10}$

Hexadecimal numbers

- Hexadecimal numbers (base-16) have 16 symbols $\{0, \dots, 9, A, B, C, D, E, F\}$.
 $(B65F)_{16} = ?$
 - $11 * 16^3 + 6 * 16^2 + 5 * 16^1 + 15 * 16^0 = (46,687)_{10}$

N-bit binary numbers

- 3-digit decimal numbers can represent 10^3 numbers: $\{000, 001, \dots, 999\}$
- 3-bit binary numbers can represent 2^3 numbers: $\{000, 001, \dots, 111\}$ i.e., $\{0, \dots, 7\}$
- A byte has 8-bits. It can represent 2^8 numbers: $\{0, \dots, 255\}$. Or $\{0, \dots, FF\}$ in hexadecimal.
 - Enough to represent a keyboard character

Kilo-Mega-Giga-Tera

- Kilo (K): 2^{10} , Mega (M): 2^{20} , Giga (G): 2^{30} , Tera (T): 2^{40}
- 1MB of file contains 2^{20} bytes = ~1 Million bytes.
How many *bits* can 1TB of disk store?

Table 1.1
Powers of Two

<i>n</i>	<i>2ⁿ</i>	<i>n</i>	<i>2ⁿ</i>	<i>n</i>	<i>2ⁿ</i>
0	1	8	256	16	65,536
1	2	9	512	17	131,072
2	4	10	1,024 (1K)	18	262,144
3	8	11	2,048	19	524,288
4	16	12	4,096 (4K)	20	1,048,576 (1M)
5	32	13	8,192	21	2,097,152
6	64	14	16,384	22	4,194,304
7	128	15	32,768	23	8,388,608

Arithmetic operations on base-r numbers

Addition

- $101101 + 100111 = ?$
- One must be careful to only allow r-allowable digits: $\{0, \dots, r - 1\}$
- When sum is $2_{10} (=10_2)$, write 0 and carry 1.
When sum is $3_{10} (=11_2)$, write 1 and carry 1
- $(101101)_2 = (45)_{10}$, $(100111)_2 = (39)_{10}$, $(1010100)_2 = (84)_{10}$

Arithmetic operations on base-r numbers

Subtraction

- $101101 - 100111 = ?$
- One must be careful to only allow r-allowable digits: $\{0, \dots, r - 1\}$
- In decimal, say 96-47.
 - When we encounter “6-7”, put (10-1) in output and reduce previous digit by 1, i.e., 9 to 8.
- In binary, 0-0 is 0. 1-1 is 0. 1-0 is 1.
When encountering 0-1: write (2-1) in output and reduce previous bit by 1.
- $(101101)_2 = (45)_{10}$, $(100111)_2 = (39)_{10}$, $(000110)_2 = (6)_{10}$

Arithmetic operations on base-r numbers

Multiplication

- $1011 * 101 = ?$
- One must be careful to only allow r-allowable digits: $\{0, \dots, r - 1\}$
- When multiplier digit is 1, copy multiplicand in the partial product. Shift multiplicand left for every bit. Compute addition of all partial products
- $(1011)_2 = (11)_{10}$, $(101)_2 = (5)_{10}$, $(110111)_2 = (55)_{10}$